

Quick Sort

快速排序

本页面将简要介绍快速排序。

定义

快速排序（英语：Quicksort），又称分区交换排序（英语：partition-exchange sort），简称「快排」，是一种被广泛运用的排序算法。

基本原理与实现

过程

快速排序的工作原理是通过 [分治](#) 的方式来将一个数组排序。

快速排序分为三个过程：

1. 将数列划分为两部分（要求保证相对大小关系）；
2. 递归到两个子序列中分别进行快速排序；
3. 不用合并，因为此时数列已经完全有序。

和归并排序不同，第一步并不是直接分成前后两个序列，而是在分的过程中要保证相对大小关系。具体来说，第一步是要把数列分成两个部分，然后保证前一个子数列中的数都小于后一个子数列中的数。为了保证平均时间复杂度，一般是随机选择一个数 来当做两个子数列的分界。

之后，维护一前一后两个指针 和 ，依次考虑当前的数是否放在了应该放的位置（前还是后）。如果当前的数没放对，比如说如果后面的指针 遇到了一个比 小的数，那么可以交换 和 位置上的数，再把 向后移一位。当前的数的位置全放对后，再移动指针继续处理，直到两个指针相遇。

其实，快速排序没有指定应如何具体实现第一步，不论是选择 的过程还是划分的过程，都有不止一种实现方法。

第三步中的序列已经分别有序且第一个序列中的数都小于第二个数，所以直接拼接起来就好了。



C++Python²



非递归实现²递归实现

```

1 struct Range {
2     int start, end;
3
4     Range(int s = 0, int e = 0) { start = s, end = e; }
5 };
6
7 template <typename T>
8 void quick_sort(T arr[], const int len) {
9     if (len <= 0) return;
10    Range r[len];
11    int p = 0;
12    r[p++] = Range(0, len - 1);
13    while (p) {
14        Range range = r[--p];
15        if (range.start >= range.end) continue;
16        T mid = arr[range.end];
17        int left = range.start, right = range.end - 1;
18        while (left < right) {
19            while (arr[left] < mid && left < right) left++;
20            while (arr[right] >= mid && left < right) right--;
21            std::swap(arr[left], arr[right]);
22        }
23        if (arr[left] >= arr[range.end])
24            std::swap(arr[left], arr[range.end]);
25        else
26            left++;
27        r[p++] = Range(range.start, left - 1);
28        r[p++] = Range(left + 1, range.end);
29    }
30 }

```

```

1 template <typename T>
2 int Partition(T A[], int low, int high) {
3     int pivot = A[low];
4     while (low < high) {
5         while (low < high && pivot <= A[high]) --high;
6         A[low] = A[high];
7         while (low < high && A[low] <= pivot) ++low;
8         A[high] = A[low];
9     }
10    A[low] = pivot;
11    return low;
12 }
13 template <typename T>
14 void QuickSort(T A[], int low, int high) {
15     if (low < high) {
16         int pivot = Partition(A, low, high);
17         QuickSort(A, low, pivot - 1);
18         QuickSort(A, pivot + 1, high);
19     }
20 }
21 template <typename T>
22 void QuickSort(T A[], int len) {
23     QuickSort(A, 0, len - 1);
24 }

```

```
1 def quick_sort(alist, first, last):
2     if first >= last:
3         return
4     mid_value = alist[first]
5     low = first
6     high = last
7     while low < high:
8         while low < high and alist[high] >= mid_value:
9             high -= 1
10        alist[low] = alist[high]
11        while low < high and alist[low] < mid_value:
12            low += 1
13        alist[high] = alist[low]
14    alist[low] = mid_value
15    quick_sort(alist, first, low - 1)
16    quick_sort(alist, low + 1, last)
```

性质

稳定性

快速排序是一种不稳定的排序算法。

时间复杂度

快速排序的最优时间复杂度和平均时间复杂度为 $O(n \log n)$ ，最坏时间复杂度为 $O(n^2)$ 。

对于最优情况，每一次选择的分界值都是序列的中位数，此时算法时间复杂度满足的递推式为 $T(n) = T(n/2) + O(n)$ ，由主定理， $T(n) = O(n \log n)$ 。

对于最坏情况，每一次选择的分界值都是序列的最值，此时算法时间复杂度满足的递推式为 $T(n) = T(n-1) + O(n)$ ，累加可得 $T(n) = O(n^2)$ 。

对于平均情况，每一次选择的分界值可以看作是等概率随机的。

► 证明

在实践中，几乎不可能达到最坏情况，而快速排序的内存访问遵循局部性原理，所以多数情况下快速排序的表现大幅优于堆排序等其他复杂度为 $O(n^2)$ 的排序算法。¹

优化

朴素优化思想

如果仅按照上文所述的基本思想来实现快速排序（或者是直接照抄模板）的话，那大概率是通不过 [P1177【模板】快速排序](#) 这道模板的。因为有毒瘤数据能够把朴素的快速排序卡成 $O(n^2)$ 。

所以，我们需要对朴素快速排序思想加以优化。较为常见的优化思路有以下三种³。

- 通过 **三数取中**（即选取第一个、最后一个以及中间的元素中的中位数）的方法来选择两个子序列的分界元素（即比较基准）。这样可以避免极端数据（如升序序列或降序序列）带来的退化；
- 当序列较短时，使用 **插入排序** 的效率更高；
- 每趟排序后，将与分界元素相等的元素聚集在分界元素周围，这样可以避免极端数据（如序列中大部分元素都相等）带来的退化。

下面列举了几种较为成熟的快速排序优化方式。

三路快速排序

定义

三路快速排序（英语：3-way Radix Quicksort）是快速排序和 [基数排序](#) 的混合。它的算法思想基于 [荷兰国旗问题](#) 的解法。

过程

与原始的快速排序不同，三路快速排序在随机选取分界点 后，将待排数列划分为三个部分：小于、等于 以及大于。这样做即实现了将与分界元素相等的元素聚集在分界元素周围这一效果。

性质

三路快速排序在处理含有多个重复值的数组时，效率远高于原始快速排序。其最佳时间复杂度为 。

实现

三路快速排序实现起来非常简单，下面给出了一种三路快排的 C++ 实现。



C++Python²

```
1 // 模板的 T 参数表示元素的类型，此类型需要定义小于 (<) 运算
2 template <typename T>
3 // arr 为需要被排序的数组，len 为数组长度
4 void quick_sort(T arr[], const int len) {
5     if (len <= 1) return;
6     // 随机选择基准 (pivot)
7     const T pivot = arr[rand() % len];
8     // i: 当前操作的元素下标
9     // arr[0, j): 存储小于 pivot 的元素
10    // arr[k, len): 存储大于 pivot 的元素
11    int i = 0, j = 0, k = len;
12    // 完成一趟三路快排，将序列分为：
13    // 小于 pivot 的元素 | 等于 pivot 的元素 | 大于 pivot 的元素
14    while (i < k) {
15        if (arr[i] < pivot)
16            swap(arr[i++], arr[j++]);
17        else if (pivot < arr[i])
18            swap(arr[i], arr[--k]);
19        else
20            i++;
21    }
22    // 递归完成对于两个子序列的快速排序
23    quick_sort(arr, j);
24    quick_sort(arr + k, len - k);
25 }
```

```

1 def quick_sort(arr, l, r):
2     if l >= r:
3         return
4     random_index = random.randint(l, r)
5     pivot = arr[random_index]
6     arr[l], arr[random_index] = arr[random_index], arr[l]
7     i = l + 1
8     j = l
9     k = r + 1
10    while i < k:
11        if arr[i] < pivot:
12            arr[i], arr[j + 1] = arr[j + 1], arr[i]
13            j += 1
14            i += 1
15        elif arr[i] > pivot:
16            arr[i], arr[k - 1] = arr[k - 1], arr[i]
17            k -= 1
18        else:
19            i += 1
20    arr[l], arr[j] = arr[j], arr[l]
21    quick_sort(arr, l, j - 1)
22    quick_sort(arr, k, r)

```

内省排序

定义

内省排序（英语：Introsort 或 Introspective sort）⁴是快速排序和 [堆排序](#) 的结合，由 David Musser 于 1997 年发明。内省排序其实是对快速排序的一种优化，保证了最差时间复杂度为 $O(n \log n)$ 。

性质

内省排序将快速排序的最大递归深度限制为 $\log n$ ，超过限制时就转换为堆排序。这样既保留了快速排序内存访问的局部性，又可以防止快速排序在某些情况下性能退化为 $O(n^2)$ 。

实现

从 2000 年 6 月起，SGI C++ STL 的 `stl_algo.h` 中 `sort()` 函数的实现采用了内省排序算法。

线性找第 k 大的数

在下面的代码示例中，第 k 大的数被定义为序列排成升序时，第 k 个位置上的数（编号从 0 开始）。

找第 k 大的数（ K -th order statistic），最简单的方法是先排序，然后直接找到第 k 大的位置的元素。这样做的时间复杂度是 $O(n \log n)$ ，对于这个问题来说很不划算。

我们可以借助快速排序的思想解决这个问题。考虑快速排序的划分过程，在快速排序的「划分」结束后，数列被分成了 L 和 R ，此时可以按照左边元素的个数 $|L|$ 和 k 的大小关系来判断是只在左边还是只在右边递归地求解。

和快速排序一样，该方法的时间复杂度依赖于每次划分时选择的分界值。如果采用随机选取分界值的方式，可以证明在期望意义下，程序的时间复杂度为 $O(n)$ 。

实现 (C++)

```

1 // 模板的 T 参数表示元素的类型，此类型需要定义小于 (<) 运算
2 template <typename T>
3 // arr 为查找范围数组，rk 为需要查找的排名（从 0 开始），len 为数组长度
4 T find_kth_element(T arr[], int rk, const int len) {
5     if (len <= 1) return arr[0];
6     // 随机选择基准 (pivot)
7     const T pivot = arr[rand() % len];
8     // i: 当前操作的元素下标
9     // arr[0, j): 存储小于 pivot 的元素
10    // arr[k, len): 存储大于 pivot 的元素
11    int i = 0, j = 0, k = len;
12    // 完成一趟三路快排，将序列分为：
13    // 小于 pivot 的元素 | 等于 pivot 的元素 | 大于 pivot 的元素
14    while (i < k) {
15        if (arr[i] < pivot)
16            swap(arr[i++], arr[j++]);
17        else if (pivot < arr[i])
18            swap(arr[i], arr[--k]);
19        else
20            i++;
21    }
22    // 根据要找的排名与两条分界线的位置，去不同的区间递归查找第 k 大的数
23    // 如果小于 pivot 的元素个数比 k 多，则第 k 大的元素一定是一个小于 pivot 的元素
24    if (rk < j) return find_kth_element(arr, rk, j);
25    // 否则，如果小于 pivot 和等于 pivot 的元素加起来也没有 k 多，
26    // 则第 k 大的元素一定是一个大于 pivot 的元素
27    else if (rk >= k)
28        return find_kth_element(arr + k, rk - k, len - k);
29    // 否则，pivot 就是第 k 大的元素
30    return pivot;
31 }

```

改进：中位数中的中位数

中位数中的中位数（英文：Median of medians），提供了一种确定性的选择划分过程中分界值的方法，从而能够让找第 k 大的数算法在最坏情况下也能实现线性时间复杂度。

该算法的流程如下：

1. 将整个序列划分为组，每组元素数不超过 5 个；
2. 寻找每组元素的中位数（因为元素个数较少，可以直接使用 [插入排序](#) 等算法）。
3. 找出这组元素中位数中的中位数。将该元素作为前述算法中每次划分时的分界值即可。

时间复杂度证明

下面将证明，该算法在最坏情况下的时间复杂度为 $O(n)$ 。设 n 为问题规模为 n 时，解决问题需要的计算量。

先分析前两步——划分与寻找中位数。由于划分后每组内的元素数量非常少，可以认为寻找一组元素的中位数的时间复杂度为 $O(1)$ 。因此找出所有 $n/5$ 组元素中位数的时间复杂度为 $O(n)$ 。

接下来分析第三步——递归过程。这一步进行了两次递归调用：第一次是寻找各组中位数中的中位数，需要的开销显然为 $O(n/5)$ ，第二次是进入分界值的左侧部分或右侧部分。根据我们选取的划分元素，有 $n/5$ 组元素的中位数小于分界值，这几组元素中，比中位数还小的元素也一定比分界值要小，从而整个序列中小于分界值的元素至少有 $n/5$ 个。同理，整个序列中大于分界值的元素也至少有 $n/5$ 个。因此，分界值的左边或右边至多有 $4n/5$ 个元素，这次递归的时间开销的上界为 $O(4n/5)$ 。

综上，我们可以列出这样的不等式：

假设 $T(n)$ 在问题规模足够小时成立。根据定义，此时有 $T(n) \leq T(n/5) + O(n)$ ，其中 c 为一正常数。将不等式右边的所有 $T(n/5)$ 进行代换：

到这里我们就证明了，该算法在最坏情况下也具有 的时间复杂度。

参考资料与注释

1. [C++ 性能榨汁机之局部性原理 - I'm Root lee !↩](#)
 2. [算法实现/排序/快速排序 - 维基教科书, 自由的教学读本↩↩↩](#)
 3. [三种快速排序以及快速排序的优化↩](#)
 4. [introsort↩](#)
-

本页面最近更新：2025/3/2 22:28:01, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [billchenchina](#), [NachtgeistW](#), [ouuan](#), [Alisahhh](#), [Backlight](#), [ChungZH](#), [Enter-tainer](#), [iamtwz](#), [Ir1d](#), [Konano](#), [ksyx](#), [mcendu](#), [Menci](#), [partychicken](#), [PhantomPower82](#), [sbofgayschool](#), [shawlleyw](#), [sshwy](#), [StudyingFather](#), [untitledunrevised](#), [weilycoder](#), [wpcwzy](#), [Xeonacid](#), [xzcxczcy](#), [zhangjunyan2580](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用